

# Personal Finance Tracker

## Project Overview

This is a complete personal finance management system that I built to practice Python! It's like having your own financial assistant that tracks every expense, generates reports, and helps you understand your spending habits. Everything gets saved to files so your financial data is always secure and accessible.

## What I Learned Building This

### 1. Object-Oriented Programming

- Creating classes to represent real-world concepts (Expense, ExpenseManager)
- Using `__init__` for object initialization
- Working with class methods and instance methods
- Understanding self and how objects store their own data
- Using `@classmethod` decorators for alternative constructors

### 2. File Handling & Data Persistence

- Reading and writing JSON files for data storage
- Creating CSV files for data export
- Building a backup and restore system
- Managing multiple directories (data, backups, exports)
- Handling file paths with `os.path.join()`

### 3. Data Validation

- Validating dates with `datetime.strptime()`
- Ensuring amounts are positive numbers
- Using try-except blocks for error handling
- Creating custom validation methods

### 4. Data Structures & Collections

- Using lists to store multiple expenses
- Working with dictionaries for data serialization
- Using `defaultdict` for automatic initialization
- List comprehensions for filtering and searching

### 5. Date & Time Operations

- Parsing date strings into date objects
- Comparing dates for range searches

- Extracting year and month for monthly reports
- Generating timestamps for unique IDs

## 6. Advanced Features

- Generating formatted reports with statistics
- Creating visual category breakdowns with ASCII bars
- Implementing search with multiple criteria
- Building a budget tracking system
- Export/import functionality

## How to Run

# Step 1: Make sure Python 3 is installed

```
python --version
```

# Step 2: Navigate to project folder

```
cd path/to/project
```

# Step 3: Create the main file (or ensure all files are present)

# Files needed: expense.py, expense\_manager.py, file\_handler.py, reports.py, main.py

# Step 4: Run the program

```
python main.py
```

# Step 5: Follow the menu options!

## Project Structure

finance-tracker/

```
├── expense.py      # Expense class with validation
├── expense_manager.py # Manages collection of expenses
├── file_handler.py  # All file operations
├── reports.py      # Report generation
├── main.py         # Main application & menu
├── data/           # Created automatically
│   ├── expenses.json # Main data file
│   └── backup/       # Backup files
```

└─ exports/ # CSV exports

## Features Breakdown

| Feature            | Description                                                             |
|--------------------|-------------------------------------------------------------------------|
| Add Expense        | Create new expense records with date, amount, category, and description |
| View All           | See all expenses sorted by date with running total                      |
| Search             | Find expenses by category, date range, or description keyword           |
| Monthly Report     | Detailed breakdown of spending for any month with percentages           |
| Category Breakdown | Visual bar chart showing spending by category                           |
| Budget Tracking    | Set budgets per category and track remaining amounts                    |
| Statistics         | View overall stats: totals, averages, highest/lowest expenses           |
| CSV Export         | Export data to spreadsheet format for external analysis                 |
| Backup/Restore     | Create timestamped backups and restore from previous versions           |

## Sample Session

=====

PERSONAL FINANCE TRACKER

=====

=====

MAIN MENU

=====

1. Add New Expense
  2. View All Expenses
  3. Search Expenses
  4. Generate Monthly Report
  5. View Category Breakdown
  6. Set/Update Budget
  7. Export Data to CSV
  8. View Statistics
  9. Backup/Restore Data
  0. Exit
- =====

Enter your choice (0-9): 1

---

## ADD NEW EXPENSE

---

Date (YYYY-MM-DD) [Enter for today]:

Amount: \$45.50

Categories:

1. Food
2. Transport
3. Shopping
4. Bills
5. Entertainment
6. Healthcare
7. Other

Select category (1-7): 1

Description: Lunch at downtown cafe

☒ Expense added successfully!

2024-12-14 | \$45.50 | Food | Lunch at downtown cafe

## Code Highlights

### Expense Class with Validation

```
class Expense:
```

```
    def __init__(self, date, amount, category, description, expense_id=None):
        self.id = expense_id if expense_id else datetime.now().strftime("%Y%m%d%H%M%S%f")
        self.date = self._validate_date(date)
        self.amount = self._validate_amount(amount)
        self.category = category
        self.description = description
```

**What I learned:** Using validation methods to ensure data integrity before storing!

### Smart Search with Multiple Criteria

```
def search_expenses(self, **criteria):
    results = self.expenses

    if 'category' in criteria:
        results = [e for e in results if e.category == criteria['category']]

    if 'date_from' in criteria:
        date_from = datetime.strptime(criteria['date_from'], "%Y-%m-%d").date()
        results = [e for e in results if e.date >= date_from]
```

**What I learned:** Using \*\*kwargs to accept flexible search parameters!

### Data Serialization

```
def to_dict(self):
    return {
        'id': self.id,
        'date': str(self.date),
        'amount': self.amount,
        'category': self.category,
        'description': self.description
    }
```

**What I learned:** Converting objects to dictionaries for JSON storage!

### Challenges I Faced & How I Solved Them

#### Challenge 1: How do I make sure expense amounts are always valid?

- **Problem:** Users might enter negative numbers, text, or zero
- **Solution:** Created `_validate_amount()` method that:
  - Converts input to float
  - Checks if positive
  - Rounds to 2 decimal places
  - Raises `ValueError` with helpful message

#### Challenge 2: How do I organize code for a larger project?

- **Problem:** One huge file becomes unmanageable
- **Solution:** Split into 5 modules:
  - `expense.py` - Data model

- expense\_manager.py - Business logic
- file\_handler.py - All I/O operations
- reports.py - Report generation
- main.py - User interface
- **Benefit:** Easy to find and modify specific functionality!

### Challenge 3: How do I prevent data loss?

- **Problem:** What if the file gets corrupted or accidentally deleted?
- **Solution:** Built a backup system that:
  - Creates timestamped backup files
  - Stores backups in separate directory
  - Allows browsing and restoring from any backup
  - Uses shutil.copy2() to preserve file metadata

### Challenge 4: How do I make reports readable and informative?

- **Problem:** Raw data is hard to understand
- **Solution:** Created multiple report types:
  - Monthly reports with totals and percentages
  - Category breakdown with ASCII bar charts
  - Statistics showing averages and extremes
  - Used string formatting: f" {amount:,.2f}" for currency

### Challenge 5: How do I search expenses flexibly?

- **Problem:** Users need to search in different ways
- **Solution:** Implemented criteria-based searching:
  - Accept any combination of filters
  - Apply filters sequentially
  - Support partial string matching for descriptions
  - Case-insensitive keyword search

### What I'd Improve Next

#### Technical Improvements

- Add input validation for dates (prevent future dates)
- Implement editing/updating existing expenses
- Add recurring expenses feature
- Create expense categories that users can customize

- Add support for multiple currencies
- Implement data encryption for security

### **Features to Add**

- Budget alerts when approaching limits
- Graphical charts using matplotlib
- Year-over-year comparisons
- Tag system for expenses
- Receipt image attachments
- Income tracking alongside expenses
- Bill reminders and due dates
- Split expenses between categories

### **User Experience**

- Add a GUI with tkinter or PyQt
- Create a web version with Flask
- Add undo/redo functionality
- Implement auto-save feature
- Add data import from bank statements
- Create interactive dashboard

### **Key Programming Concepts Applied**

#### **1. Separation of Concerns**

Each class has a single, clear responsibility:

- Expense - Represents individual expense
- ExpenseManager - Manages collection
- FileHandler - Handles persistence
- ReportGenerator - Creates reports
- FinanceTracker - User interface

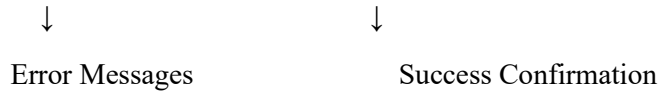
#### **2. Error Handling**

Used try-except throughout for:

- File operations (IOError)
- Date parsing (ValueError)
- JSON operations (JSONDecodeError)
- User input validation

### 3. Data Flow

User Input → Validation → Expense Object → Manager → File Handler → JSON File



### 4. Code Reusability

- Created reusable validation methods
- Used helper functions like `_ensure_directories()`
- Leveraged to `_dict()` and `from_dict()` for serialization

#### Files Created

- **expense.py** - Expense class with validation logic
- **expense\_manager.py** - Expense collection management
- **file\_handler.py** - All file I/O operations
- **reports.py** - Report generation and statistics
- **main.py** - Main application and user interface
- **data/expenses.json** - Automatically created for data storage
- **data/backup/** - Timestamped backup files
- **data/exports/** - CSV export files

#### Sample Data File (expenses.json)

```
[
  {
    "id": "20241214143055123456",
    "date": "2024-12-14",
    "amount": 45.5,
    "category": "Food",
    "description": "Lunch at downtown cafe"
  },
  {
    "id": "20241214143100234567",
    "date": "2024-12-13",
    "amount": 120.0,
    "category": "Bills",
    "description": "Internet bill"
  }
]
```



}

]

### What Makes This Project Special

1. **Real-world application** - Actually useful for tracking personal finances
  2. **Complete CRUD operations** - Create, Read, Update (via search), Delete
  3. **Data persistence** - Information saved between sessions
  4. **Professional structure** - Modular design like real software
  5. **Error handling** - Graceful failure and helpful messages
  6. **Multiple output formats** - JSON for storage, CSV for export
  7. **Comprehensive reporting** - Multiple ways to analyze data
  8. **Backup system** - Protection against data loss
-